

The Oulipian text generator I created replaces the last word of each sentence with the 7th most likely next predicted word. And for the P+x version, which is achieved by modifying the following lines:

```
“seventh_token_idx = top_k_indices[6]”
```

```
“seventh_token_prob = top_k_probs[6]”
```

You change the number “6” to any number you wish, and the program will replace the last word of each line of the poem with the prediction assigned to that rank. The program ranks predictions by order of likeability. The likeability of a prediction is determined by the algorithm’s understanding of the sentence, and its interpretation of what word would be the most likely to follow the last word in the sentence. Consequently, when you create a P+x variant, the higher the value of x, the less contextually viable the words you end up with are. Which make your resulting output increasingly weirder and illogical. In my case, when I made my P+20 variant, the word “snow” was replaced with “and have been cold a long”. Which is quite unusual, but interesting. So, I settled with the 20th prediction.

If I wanted to create a variant of my code that would implement a P+7 technique in which all nouns are replaced with their seventh-highest probability alternatives, I would make the program perform word-level tokenization and identify each word token that belongs to a noun and then perform the actions that the original program would perform, which are the probability check, grab the word’s 7th most likely alternative, and replace that word with it, but for each detected noun instead.